

Build de Imagem Docker

Execução de Containers e Prática com Node.js

Objetivos da Aula

Ao final da aula o aluno deverá ser capaz de:

- Compreender a diferença entre **Imagem** e **Container**
- Escrever um **Dockerfile** para uma aplicação Node.js
- **Construir** uma imagem Docker com `docker build`
- **Executar** containers com `docker run`
- **Containerizar** uma aplicação Node.js do zero

Revisão: Imagem vs Container

Conceito	Descrição
Imagem	Receita/molde estático (somente leitura)
Container	Instância em execução de uma imagem
Dockerfile	Arquivo de instruções para criar a imagem
Docker Hub	Repositório público de imagens



Imagem é o pacote. Container é o processo rodando.

O que é um Dockerfile?

O **Dockerfile** é um arquivo de texto com instruções que o Docker usa para construir uma imagem automaticamente.

```
Dockerfile → docker build → Imagem → docker run → Container
```

- **Reprodutível:** Qualquer pessoa gera a mesma imagem
- **Versionável:** Pode ser commitado junto com o código
- **Automatizável:** Usado em pipelines de CI/CD

Anatomia de um Dockerfile

```
# 1. Imagem base
FROM node:20-alpine

# 2. Diretório de trabalho dentro do container
WORKDIR /app

# 3. Copia os arquivos de dependências
COPY package*.json ./

# 4. Instala as dependências
RUN npm install

# 5. Copia o restante do código
COPY . .

# 6. Expõe a porta da aplicação
EXPOSE 3000

# 7. Comando padrão ao iniciar o container
CMD ["node", "server.js"]
```

Instruções Principais do Dockerfile

Instrução	Função
FROM	Define a imagem base
WORKDIR	Define o diretório de trabalho
COPY	Copia arquivos do host para o container
RUN	Executa comandos durante o build
EXPOSE	Documenta a porta que a app usa
ENV	Define variáveis de ambiente
CMD	Comando padrão ao iniciar o container
ENTRYPOINT	Ponto de entrada fixo do container

Construindo uma Imagem: `docker build`

```
# Sintaxe básica  
docker build -t nome-da-imagem:tag .
```

```
# Exemplos  
docker build -t minha-app:1.0 .  
docker build -t minha-app:latest .
```

```
# Verificar imagens criadas  
docker images
```

- `-t` → Define o nome (tag) da imagem
- `.` → Contexto de build (diretório atual)
- A tag `:latest` é aplicada por padrão se omitida

Executando um Container: `docker run`

Sintaxe básica

```
docker run [opções] nome-da-imagem
```

Exemplos práticos

```
docker run minha-app:latest
```

Mapear porta: host:container

```
docker run -p 3000:3000 minha-app:latest
```

Em background (modo detached)

```
docker run -d -p 3000:3000 minha-app:latest
```

Com nome definido

```
docker run -d -p 3000:3000 --name meu-servidor minha-app:latest
```

Comandos Docker Essenciais

```
# Listar containers em execução  
docker ps
```

```
# Listar todos os containers (incluindo parados)  
docker ps -a
```

```
# Parar um container  
docker stop meu-servidor
```

```
# Remover um container  
docker rm meu-servidor
```

```
# Ver logs do container  
docker logs meu-servidor
```

```
# Acessar o terminal do container  
docker exec -it meu-servidor sh
```

Prática: App Node.js com Express

Vamos containerizar uma API simples passo a passo.

Estrutura do projeto

```
minha-api/  
├── server.js  
├── package.json  
└── Dockerfile
```

Pré-requisitos

- Docker instalado e rodando
- Node.js instalado (apenas para iniciar o projeto localmente)

Passo 1: Criar o projeto Node.js

```
mkdir minha-api && cd minha-api  
npm init -y  
npm install express
```

server.js

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;

app.get("/", (req, res) => {
  res.json({
    mensagem: "API rodando em um container Docker! 🐳",
    versao: "1.0.0",
    ambiente: process.env.NODE_ENV || "development",
  });
});

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

Passo 2: Criar o Dockerfile

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install --omit=dev

COPY . .

EXPOSE 3000

CMD ["node", "server.js"]
```

Por que `node:20-alpine`?

- **Alpine Linux:** imagem minimalista (~5 MB)
- **Resultado:** imagem final muito menor
- **Segurança:** menor superfície de ataque

Passo 3: Criar o .dockerignore

Evita copiar arquivos desnecessários para o container:

```
node_modules  
npm-debug.log  
.git  
.gitignore  
*.md  
.env
```

Funciona igual ao `.gitignore`, mas para o Docker.

Reduz o tamanho do contexto de build e da imagem final.

Passo 4: Build e Execução

```
# 1. Construir a imagem
docker build -t minha-api:1.0 .

# 2. Verificar se a imagem foi criada
docker images

# 3. Executar o container
docker run -d -p 3000:3000 --name api minha-api:1.0

# 4. Verificar se está rodando
docker ps

# 5. Testar a API
curl http://localhost:3000
# ou abra no navegador: http://localhost:3000
```

Passo 5: Inspecionar e Parar

```
# Ver logs em tempo real
docker logs -f api

# Entrar no terminal do container
docker exec -it api sh

# Parar o container
docker stop api

# Remover o container
docker rm api

# Remover a imagem (se necessário)
docker rmi minha-api:1.0
```

Boas Práticas com Dockerfile

-  Use imagens base **oficiais e leves** (`alpine` , `slim`)
-  Copie o `package.json` **antes** do código (cache de camadas)
-  Use `--omit=dev` para não instalar dependências de desenvolvimento
-  Sempre crie um `.dockerignore`
-  Um processo por container
-  Não rode a aplicação como root (use `USER node`)
-  Não coloque segredos (senhas, tokens) no Dockerfile

Resumo do Fluxo Completo

1. Escrever código (server.js)
↓
2. Definir dependências (package.json)
↓
3. Criar Dockerfile + .dockerignore
↓
4. `docker build -t minha-api:1.0 .`
↓
5. `docker run -d -p 3000:3000 minha-api:1.0`
↓
6. Testar em `http://localhost:3000`

Com um Dockerfile versionado no repositório, qualquer desenvolvedor ou pipeline de CI/CD pode reproduzir o mesmo ambiente.

Perguntas?